

TrueSpec Automotive — Secure Inventory Website (Build Spec)

Overview

Build a two-sided web application for TrueSpec Automotive, a vehicle import business:

1. **Public site** — a polished, customer-facing showcase of vehicle inventory that anyone can browse.
2. **Admin backend** — a private, authenticated dashboard where only the business owner can add, edit, and manage inventory, including sensitive cost and profit data.

The critical requirement: **sensitive financial data must never be retrievable by the public, under any circumstance** — not through the UI, not through the API, not through browser dev tools. This must be enforced on the server, not just hidden in the frontend.

Users & Roles

- **Admin (single user)**: the business owner. Full access to create/edit/delete listings, view cost breakdowns and profit, manage settings.
 - **Public visitor**: anonymous. Can only browse listings and view the public-facing fields below. No login required, no write access of any kind.
-

Data Model

Split every vehicle listing into two tiers. This split must exist at the database and API level — not just in what the frontend chooses to render.

Public fields (visible to everyone)

- Brand, model, trim, year
- Exterior color, interior color
- Mileage

- Notable features / options (free text or tags)
- Photos (multiple, gallery-style)
- Listing status: `Available` , `On Order` , or `Landed` (this year)
- Customer-facing price ("doorstep price" — the number the customer pays, not internal cost)
- Optional note (e.g. expected arrival date for on-order vehicles)

Admin-only fields (never exposed via any public endpoint)

- Purchase price
- Inland trucking cost (USA)
- Shipping cost
- Clearing cost
- Inland trucking cost (Nigeria)
- Fixed "full tank" cost (₦110,000 default, editable)
- Computed total landed cost
- Computed profit (customer price minus landed cost)
- Internal notes / sourcing contact info

Implementation requirement: the public API must use an explicit allow-list serializer for the public fields — never a "return everything, strip fields on the client" approach. If a field isn't explicitly allow-listed for the public response, it should not be possible for it to leak through, even by mistake in a future code change.

Authentication & Security

- Real, server-verified authentication for the admin (email + password, or a magic-link/passwordless flow) — session or token-based, checked on every single admin API request. No client-side-only PIN or password check.
- All admin routes (API and any admin UI pages) must reject unauthenticated requests at the server, returning 401/403 — never relying on the frontend to simply "not show" the admin UI.
- Rate-limit login attempts.

- Store only hashed credentials (e.g. bcrypt/argon2) if using password auth — never plaintext.
 - All admin actions (create/edit/delete) should be logged with a timestamp, so the owner can audit changes.
-

Functional Requirements

Public site

- Home page with hero section, logo, tagline, and a prominent "Chat on WhatsApp" button (deep link with a pre-filled message).
- Three browsable sections/tabs: **Available Inventory**, **On Order**, **Landed** (this year) — each its own filtered list.
- Each listing displays as a card (photo, brand/model/trim/year, colors, mileage, price) that expands into a detail view with the full photo gallery and features.
- A WhatsApp button on every individual listing, pre-filled with a message naming that specific vehicle.
- Fully responsive: works cleanly on both desktop and mobile browsers (not just "shrinks," but reflows navigation, grids, and typography appropriately at each breakpoint).

Admin dashboard

- Secure login.
 - Full CRUD on listings across all three statuses (Available / On Order / Landed).
 - Multi-photo upload per listing (drag-and-drop or multi-select), with automatic image compression/resizing on upload.
 - Cost breakdown entry form (all admin-only fields above) with a live-calculated landed cost and profit shown as you type.
 - A summary view: total inventory count, total landed cost across inventory, total projected profit.
 - Settings page: WhatsApp number, site tagline, fixed "full tank" cost default.
-

Design

- Dark, elegant automotive-showroom aesthetic — deep charcoal/graphite background, one warm gold accent color for primary actions, WhatsApp-green for the chat CTA.
 - TrueSpec logo top-left, consistent across public and admin views (logo file to be provided separately).
 - Condensed/industrial display typeface for headings (e.g. Oswald), clean sans-serif for body text (e.g. Inter).
 - Subtle hover/elevation effects on desktop; touch-friendly tap targets on mobile.
-

Non-Functional Requirements ("no bugs" in practice)

Since no software is bug-proof, treat this as a rigor checklist rather than a guarantee:

- Use a typed language/framework (e.g. TypeScript) to catch data-shape errors at build time.
 - Validate and sanitize all inputs server-side (not just client-side form validation).
 - Handle every async operation's error state explicitly — no silent failures; show the user clear feedback if a save, upload, or fetch fails, with automatic retry for transient network issues.
 - Write automated tests for the highest-risk paths specifically:
 - Public API never returns admin-only fields, even when requested directly.
 - Unauthenticated requests to admin routes are rejected.
 - Creating/editing a listing twice in quick succession does not create duplicate records.
 - Image upload handles large batches without corrupting or duplicating files.
 - Test on both a real desktop browser and a real mobile browser before launch, not just responsive-mode in dev tools.
-

Suggested Stack (adjust to your builder's preference)

- Frontend: Next.js (React) or similar SSR-capable framework
- Backend: Next.js API routes / a small Node.js (Express) service, or a BaaS like Supabase with row-level security enforcing the public/admin split at the database level

- Database: PostgreSQL
 - Image storage: Supabase Storage, Cloudinary, or S3
 - Auth: Supabase Auth, NextAuth, or Clerk
 - Hosting: Vercel, Railway, or Render
-

Deliverable

A live, deployed website with:

1. A public URL for customer browsing (shareable in Instagram bio / WhatsApp).
2. A separate `/admin` login route for the owner only.
3. Documentation (even brief) on how to log in, add a listing, and update settings.